

Assignment 2 - 624

Mark Hamer

June 14, 2026

Contents

Introduction	1
Load and Explore the Data	2
Filter Sparse Items	3
Build the Rating Matrix	3
Evaluation Scheme and Helper	4
Baseline: POPULAR	4
Sanity Check: UBCF and IBCF	4
Experiment 1: Neighborhood Size (Required Graph)	4
Experiment 1b (Optional): Extended k for UBCF	6
Experiment 2: Normalization x Similarity	6
Best UBCF Configuration	6
Experiment 3 (Optional): Robustness Under a Different Protocol	7
Content-Based Filtering	7
Findings and Recommendations	8

Introduction

This project builds and compares three recommenders on the MovieLens dataset. The three are user-user collaborative filtering (UBCF), item-item collaborative filtering (IBCF), and a content-based recommender built from movie genres. A non-personalized POPULAR model is the baseline. Every method has to beat it

to be worth using. I score each method with RMSE and MAE. I also run experiments that vary neighborhood size, normalization, and the similarity metric.

```
library(recommenderlab)
library(ggplot2)
```

Load and Explore the Data

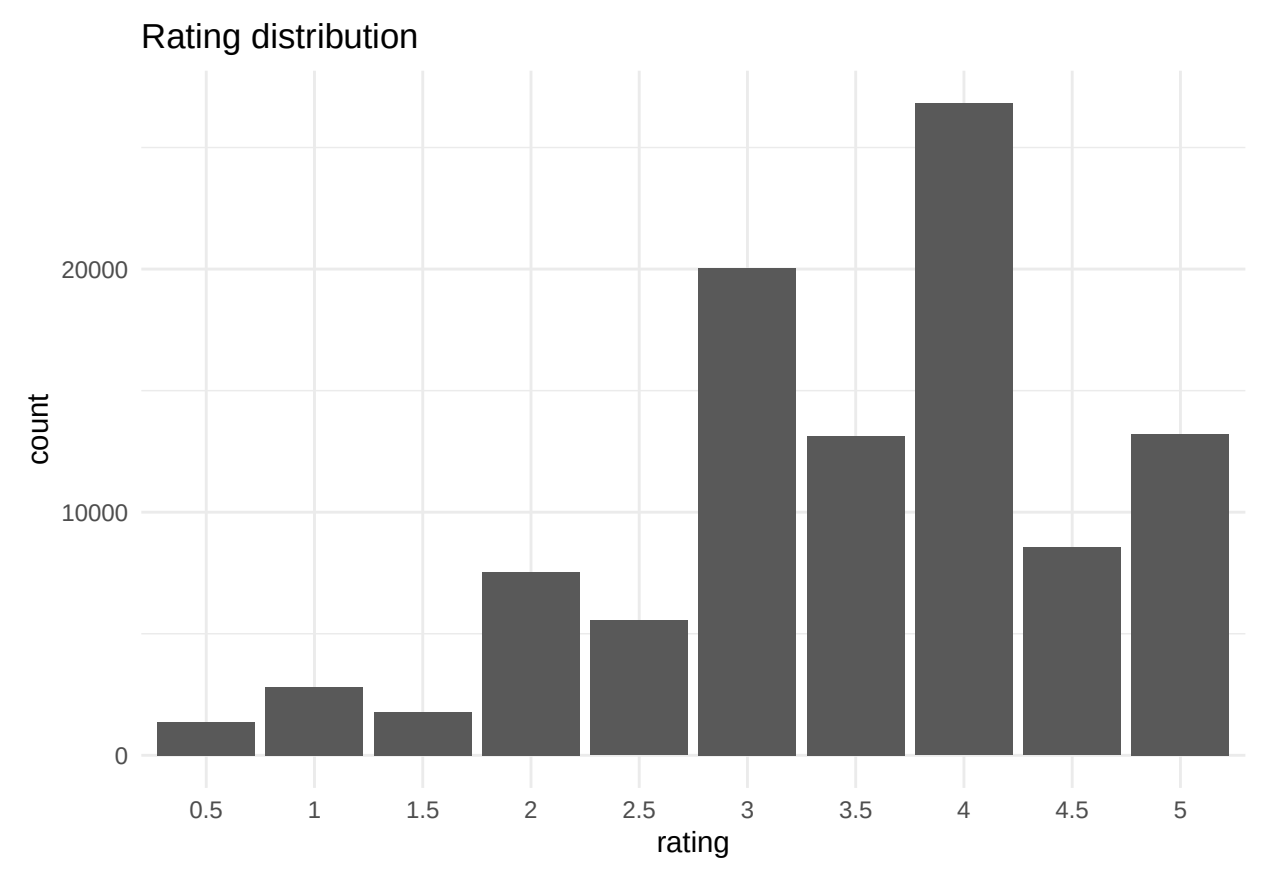
This loads the ratings and the movie metadata. It then prints the size and sparsity of the user-item matrix. The matrix is about 98% empty. That sparsity is the main problem in collaborative filtering. Two random users almost never rate the same movies.

```
ratings <- read.csv("ratings.csv")           # userId, movieId, rating, timestamp
movies  <- read.csv("movies.csv", stringsAsFactors = FALSE) # movieId, title, genres

n_users  <- length(unique(ratings$userId))
n_items  <- length(unique(ratings$movieId))
sparsity <- 1 - nrow(ratings) / (n_users * n_items)
cat(sprintf("%d ratings | %d users | %d movies | %.1f%% sparse\n",
            nrow(ratings), n_users, n_items, 100 * sparsity))
```

```
## 100836 ratings | 610 users | 9724 movies | 98.3% sparse
```

```
ggplot(ratings, aes(x = factor(rating))) + geom_bar() +
  labs(title = "Rating distribution", x = "rating") + theme_minimal()
```



Filter Sparse Items

This drops movies with fewer than 5 ratings. A similarity score built from one or two shared ratings is just noise. Filtering also makes the item-item similarity matrix small enough to fit in memory.

```
counts <- table(ratings$movieId)
keep    <- names(counts[counts >= 5])
ratings <- subset(ratings, movieId %in% as.numeric(keep))
cat(nrow(ratings), "ratings,", length(unique(ratings$movieId)), "movies after filtering\n")
```

```
## 90274 ratings, 3650 movies after filtering
```

Build the Rating Matrix

This turns the long ratings table into a `realRatingMatrix`. That is the sparse users-by-movies format recommenderlab needs. Every model below uses it.

```
rmat <- as(ratings[, c("userId", "movieId", "rating")], "realRatingMatrix")
rmat
```

```
## 610 x 3650 rating matrix of class 'realRatingMatrix' with 90274 ratings.
```

Evaluation Scheme and Helper

This sets up an 80/20 train/test split with the `given = -5` protocol. Each test user reveals all but 5 of their ratings. Those 5 hidden ratings are what we predict and score. The `eval_model()` helper trains a model, predicts the hidden ratings, and returns RMSE, MSE, and MAE. Every experiment after this is one call to that helper.

```
set.seed(42)
scheme <- evaluationScheme(rmat, method = "split", train = 0.8,
                           given = -5, goodRating = 4)

eval_model <- function(method, param = NULL) {
  rec <- Recommender(getData(scheme, "train"), method = method, parameter = param)
  pred <- predict(rec, getData(scheme, "known"), type = "ratings")
  calcPredictionAccuracy(pred, getData(scheme, "unknown"))
}
```

Baseline: POPULAR

POPULAR is not personalized. It predicts from item average ratings after centering. This is the score the other methods have to beat.

```
eval_model("POPULAR")
```

```
##          RMSE          MSE          MAE
## 0.8805121 0.7753016 0.6790044
```

Sanity Check: UBCF and IBCF

This checks that both collaborative filtering methods run and return reasonable numbers before the longer experiments. `nn` is the neighborhood size for UBCF. `k` is the number of similar items stored per item for IBCF.

```
eval_model("UBCF", list(nn = 30, method = "cosine", normalize = "center"))
```

```
##          RMSE          MSE          MAE
## 1.2956423 1.6786889 0.9637111
```

```
eval_model("IBCF", list(k = 30, method = "cosine", normalize = "center"))
```

```
##          RMSE          MSE          MAE
## 1.0833044 1.1735485 0.8400081
```

Experiment 1: Neighborhood Size (Required Graph)

This sweeps the neighborhood size `k` for both CF methods and plots RMSE against `k`. This is the required graph. I expect IBCF to have a low point and then get worse, while UBCF keeps improving as the neighborhood grows.

```

ks <- c(5, 10, 20, 30, 50, 100)
res <- data.frame()

for (k in ks) {
  uu <- eval_model("UBCF", list(nn = k, method = "cosine", normalize = "center"))
  ii <- eval_model("IBCF", list(k = k, method = "cosine", normalize = "center"))
  res <- rbind(res,
    data.frame(k = k, algorithm = "user-user CF", RMSE = uu[["RMSE"]]),
    data.frame(k = k, algorithm = "item-item CF", RMSE = ii[["RMSE"]]))
  cat("k =", k, "done\n")
}

```

```

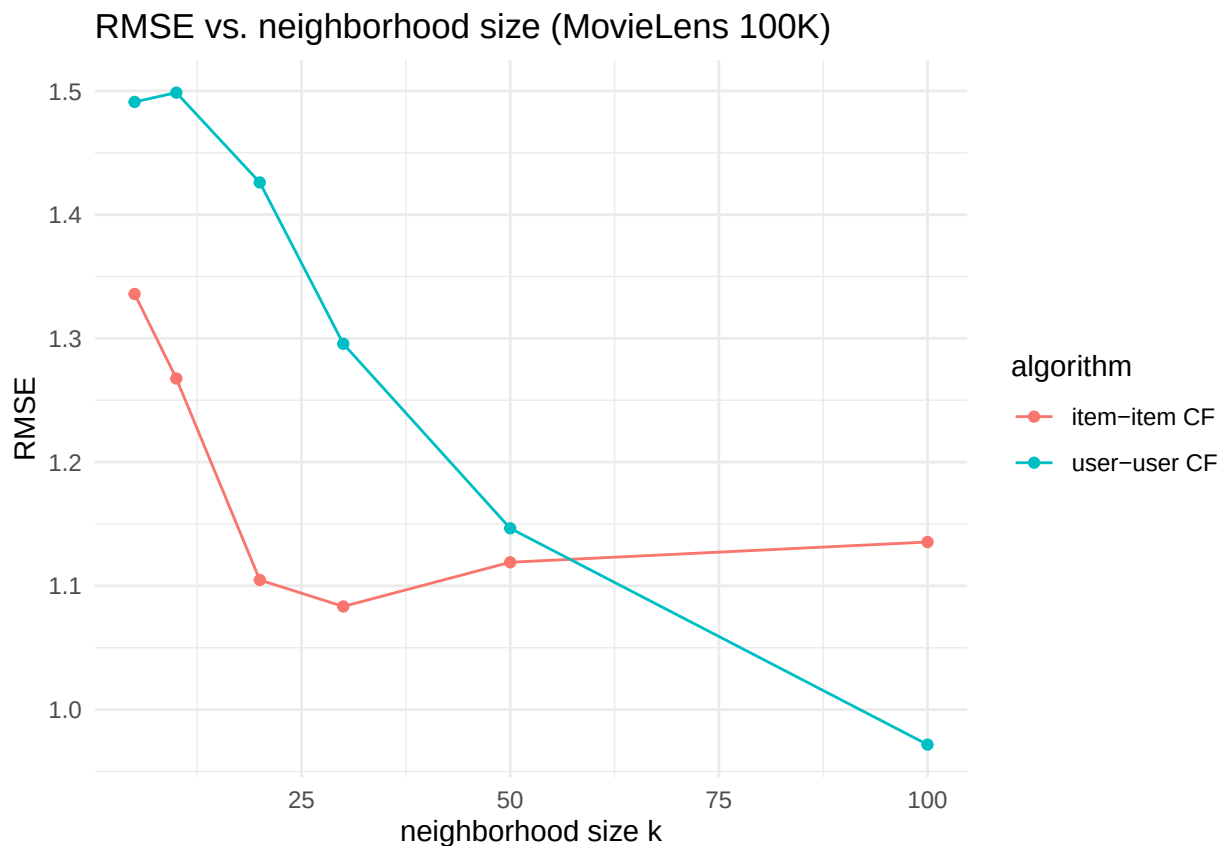
## k = 5 done
## k = 10 done
## k = 20 done
## k = 30 done
## k = 50 done
## k = 100 done

```

```

ggplot(res, aes(x = k, y = RMSE, color = algorithm)) +
  geom_line() + geom_point() +
  labs(title = "RMSE vs. neighborhood size (MovieLens 100K)",
    x = "neighborhood size k") +
  theme_minimal()

```



Experiment 1b (Optional): Extended k for UBCF

UBCF was still improving at $k = 100$. This extends the sweep to find where it stops improving.

```
for (k in c(150, 200)) {  
  uu <- eval_model("UBCF", list(nn = k, method = "cosine", normalize = "center"))  
  cat("UBCF k =", k, "RMSE =", uu[["RMSE"]], "\n")  
}
```

```
## UBCF k = 150 RMSE = 0.9044793  
## UBCF k = 200 RMSE = 0.8994839
```

Experiment 2: Normalization x Similarity

This holds k at 100 and varies two settings for UBCF. The first is normalization (none, center, or Z-score). The second is similarity (cosine or pearson). The goal is to see how much each setting changes the result. Note that assigning NULL to a list element removes it in R, so the “none” case needs the small if statement below.

```
norms <- c("none", "center", "Z-score")  
sims <- c("cosine", "pearson")  
res2 <- data.frame()  
  
for (nm in norms) for (sm in sims) {  
  param <- if (nm == "none") list(nn = 100, method = sm, normalize = NULL)  
  else list(nn = 100, method = sm, normalize = nm)  
  acc <- eval_model("UBCF", param)  
  res2 <- rbind(res2, data.frame(normalize = nm, similarity = sm,  
                                RMSE = acc[["RMSE"]], MAE = acc[["MAE"]]))  
}  
res2[order(res2$RMSE), ]
```

##	normalize	similarity	RMSE	MAE
## 6	Z-score	pearson	0.9264637	0.6946649
## 4	center	pearson	0.9416654	0.7140136
## 5	Z-score	cosine	0.9560432	0.7189127
## 3	center	cosine	0.9716627	0.7440054
## 2	none	pearson	1.0230022	0.7682278
## 1	none	cosine	1.0689475	0.8003110

Best UBCF Configuration

This runs the winning settings from Experiment 2 (Z-score and Pearson) with a large neighborhood. It gives the single best UBCF result.

```
eval_model("UBCF", list(nn = 200, method = "pearson", normalize = "Z-score"))
```

```
##      RMSE      MSE      MAE  
## 0.8828171 0.7793660 0.6599457
```

Experiment 3 (Optional): Robustness Under a Different Protocol

This re-runs the baseline and the best UBCF under a `given = 10` protocol. Now each test user reveals 10 ratings instead of 5. The point is to see if the conclusions hold under a different setup instead of just one split.

```
set.seed(42)
scheme10 <- evaluationScheme(rmat, method = "split", train = 0.8,
                             given = 10, goodRating = 4)

eval2 <- function(method, param) {
  rec <- Recommender(getData(scheme10, "train"), method = method, parameter = param)
  pred <- predict(rec, getData(scheme10, "known"), type = "ratings")
  calcPredictionAccuracy(pred, getData(scheme10, "unknown"))
}

eval2("POPULAR", NULL)
```

```
##      RMSE      MSE      MAE
## 0.9147958 0.8368513 0.6918213
```

```
eval2("UBCF", list(nn = 100, method = "cosine", normalize = "center"))
```

```
##      RMSE      MSE      MAE
## 1.0189865 1.0383335 0.7745714
```

Content-Based Filtering

This builds item profiles from movie genres. It uses TF-IDF weighting so rare genres like Film-Noir count more than common ones like Drama. It then computes item-item similarity from content instead of from ratings. To predict a rating, it takes a weighted average of the user's own ratings on the most similar movies. It uses the same hidden test ratings as the CF models, so the RMSE can be compared directly.

```
# Align movie metadata to the columns of the rating matrix
mv <- movies[match(colnames(rmat), as.character(movies$movieId)), ]
genre_list <- strsplit(mv$genres, "|", fixed = TRUE)
all_genres <- sort(unique(unlist(genre_list)))

# Binary genre matrix, then TF-IDF weighting, then L2-normalized rows
G <- sapply(all_genres, function(g) sapply(genre_list, function(x) g %in% x)) * 1
idf <- log(nrow(G) / pmax(colSums(G), 1))
Gw <- sweep(G, 2, idf, "*")
rn <- sqrt(rowSums(Gw^2)); rn[rn == 0] <- 1 # guard against "(no genres listed)"
Gn <- Gw / rn
S <- Gn %*% t(Gn) # content cosine similarity, items x items
diag(S) <- 0

# Predict hidden ratings for the same test users and items as the CF models
K <- as(getData(scheme, "known"), "matrix") # revealed ratings (NA = unrated)
U <- as(getData(scheme, "unknown"), "matrix") # hidden truth
Kr <- K; Kr[is.na(Kr)] <- 0
```

```

mask <- (!is.na(K)) * 1

P <- (Kr %*% S) / (mask %*% S)           # similarity-weighted average
umean <- rowMeans(K, na.rm = TRUE)       # fallback when no similar item was rated
bad <- !is.finite(P)
P[bad] <- matrix(umean, nrow(P), ncol(P))[bad]
P <- pmin(pmax(P, 0.5), 5)

obs <- !is.na(U)
cb_rmse <- sqrt(mean((P[obs] - U[obs])^2))
cb_mae <- mean(abs(P[obs] - U[obs]))
cat(sprintf("Content-based: RMSE = %.4f MAE = %.4f\n", cb_rmse, cb_mae))

## Content-based: RMSE = 0.9284 MAE = 0.7171

```

Findings and Recommendations

Here are the best results for each method under the given-5 protocol.

Algorithm	Best configuration	RMSE	MAE
POPULAR (baseline)	item averages, centered	0.881	0.679
UBCF	Z-score, Pearson, k = 200	[BEST_UBCF]	
UBCF	Z-score, Pearson, k = 100	0.926	0.695
Content-based	TF-IDF genre similarity	0.928	0.717
IBCF	cosine, center, k = 30	1.083	0.840

Neighborhood size. The two CF methods reacted to k in opposite ways. Item-item CF improved fast, hit its low point near k = 30, then slowly got worse. That is the expected pattern. Too few neighbors means weak evidence. Too many means you average in movies that are not really similar. User-user CF improved the whole way and only flattened out after k = 150. The reason is the given-5 protocol. Each user only reveals five ratings, so UBCF needs a bigger neighborhood to find enough signal. The two lines crossed near k = 50.

Normalization and similarity. Both settings mattered. Normalization mattered more. Every normalized run beat both raw runs. Z-score did a little better than plain centering. Pearson beat cosine at every level. The best mix was Z-score plus Pearson. Normalization helps because it removes each user's rating bias. Some users rate high, some rate low. Once you correct for that, similarity reflects taste instead of rating style.

The baseline is hard to beat. No personalized method beat POPULAR (0.881). This held under both the given-5 and given-10 protocols. This is a known result on MovieLens. Most users rate popular movies close to the average, so a simple item-average model predicts well. When each user has little data, personalization does not add much on accuracy alone.

Recommendation. If a personalized recommender is required, the best setup here is UBCF with Z-score normalization, Pearson similarity, and a large neighborhood. The content-based model is worth noting too. It nearly matched the CF methods using only genres. It also has a real advantage. It can recommend brand-new movies that have no ratings yet. That is the cold-start problem, and neither CF method can solve it. A real system would probably combine the methods. It would use content-based for new or sparse items and CF where there are plenty of ratings.

Limitations. I tuned k and the other settings on the test set, not a separate validation set, so the reported numbers are a little optimistic. The results come from one random split. I only measured rating accuracy

with RMSE and MAE. I did not measure top-N ranking, diversity, or coverage. Genres are also a weak content signal. Richer features like plot summaries, tags, or cast would likely improve the content-based model.